

GUADE Framework

Copyright (c) 2026 LeadRescue LLC. All rights reserved.

Purpose

GUADE is a real local-first workflow agent framework. A workflow is executable infrastructure. Agents receive explicit instructions, explicit tool grants, upstream handoffs, and a run workspace. Every run leaves a ledger entry and artifacts on disk.

Core Contract

- Agents do real model calls through an OpenAI-compatible Responses API.
- Workflows are directed step graphs defined in JSON.
- A step runs only after its declared dependencies complete.
- Upstream outputs are passed into downstream prompts as handoff material.
- Tools are granted per agent, not globally.
- Filesystem tools are scoped to the run workspace.
- Shell execution is disabled unless the operator explicitly passes `--allow-shell`.
- Every prompt, output, tool event, workflow definition, and task is persisted.

Runtime Pieces

1. CLI entrypoint

```
python -m workflow_agent run <workflow.json> "<task>"
```

2. Workflow loader

Parses agents, steps, dependencies, tool grants, and model overrides. It rejects invalid graphs before any model call starts.

3. Runner

Creates a run ID and workspace, executes steps in graph order, builds prompts from the user task and dependency outputs, invokes the model provider, and writes step artifacts.

4. Provider

Calls the OpenAI-compatible `/v1/responses` endpoint. It supports custom function tools and loops until the model returns final text or the tool-round limit is reached.

5. Tool runtime

Exposes real functions: list, read, write, fetch URL, and optionally shell. Tool calls are logged into each step's metadata.

6. Ledger

SQLite database under `GUADE_HOME`, defaulting to `~/.guade`, plus a run folder containing prompts, outputs, metadata, and generated artifacts.

7. MCP server

A stdio MCP adapter exposes GUADE to Claude Code in the terminal. Claude can list workflows, run workflows, list prior runs, and inspect run outputs.

8. Dashboard

A local HTTP service exposes the same workflow runner and ledger to the browser. The command center launches background runs, polls their real status, and displays saved step outputs and generated workspace artifacts.

9. Connector runtime

Workflow agents can receive run-scoped tools from local stdio or remote HTTP MCP servers. Connector definitions contain commands, endpoints, and environment-variable references; secret values stay in the GUADE process environment.

10. Income and creator workbench

Opportunity and shop setup workflows produce reviewable launch packages. The local creator studio trims video segments and saves rendered thumbnails under the GUADE data directory.

11. Provider routing and operator assistant

Workflow agents use Ollama's OpenAI-compatible Responses API or the OpenAI API, including function tools. Direct operator chat can additionally use the installed Codex CLI in an ephemeral read-only workspace. Provider and model preferences are stored without credentials; API keys remain in the service environment.

12. Payments and business identity connectors

The dashboard categorizes generic MCP connections for payments, banking, crypto wallet tools, and business identity workflows. GUADE does not custody keys or store bank credentials, EINs, or D-U-N-S numbers, and does not execute payments itself.

Current File Layout

```
workflow-agent/  
  workflow_agent/  
    cli.py  
    creator_media.py  
    models.py
```

```
mcp_connectors.py
provider.py
runner.py
storage.py
tool_runtime.py
tools.py
workflow.py
mcp_server.py
bin/
  guade-mcp
  guade-desktop
  install-guade-desktop
  workflow-agent-mcp
web/
  index.html
  app.css
  app.js
  chat.css
  operations.css
  grants.css
examples/
  income_opportunity_scan.json
  competitor_intelligence.json
  local_builder.json
  research_build.json
  shop_setup_kit.json
  sales_output.json
docs/
  architecture.md
tests/
  test_workflow.py
```

Next Engineering Milestones

- Add human approval queues for shell/network/write operations.
- Add connector tools for GitHub, Gmail, Drive, browser search, and databases.
- Add concurrent execution for independent branches.
- Add skill packages so agents can import repeatable procedures.
- Add cost normalization per provider and per workflow.